

LAB 4/GROUP 1&2: REPRESENTATION OF GRAPHS

TASK 1: *Take from moodle the classes `Canvas` and `Application` and include them in your package `<yourLastNames>`.*

- a) Change package names in both classes to `<yourLastNames>`.
- b) A class `Canvas` is provided which contains the `main`-method and which can be extended if further user-interface stuff will be implemented (optionally).
- c) A second class `Application` is provided that is the entry-point of your application and will contain an instance of your graph (see Task 2).
In this class, an example is given, how a line and circles can be drawn on the screen. Use such instructions in order to visualise your graph via using appropriate getters in class `Graph`.

TASK 2: *Write a program which represents a graph as an adjacency list (Group1) and an adjacency matrix (Group2).*

- a) Provide a class `Graph` which contains as a property the adjacency list (Group1) or adjacency matrix (Group2).
- b) Implement the UML diagram on the last page (Fig. 1).
- c) The vertices are represented by a class `Node`. Think about what a node should contain.
- d) In the class `Graph` a method `somePath(int u, int length)` is to be provided, which follows an arbitrary path from vertex `u` which has the provided `length`. Invent a simple algorithm for generating such a path. The resulting path is to be stored in an object of class `AdjacencyList` and visualised on the screen by color.

Each group has to define an own path and implementation of `somePath(int u, int length)`. There are infinite ways to do so. This method is the main challenge of this last exercise. Visualise your final path.

PROGRAMMING STYLE RULES

- a) Use the programming language Java.
- b) Denote package names lowercase.
- c) Use the following conventions in your programs:
 - *english* names
 - meaningful, but short names
 - camelCase
 - classes, interfaces, static variables start uppercase
 - variables start lowercase
 - constants are completely uppercase
- d) further code conventions
 - in general, declare everything as to be private; only where necessary, protected or public can be used for methods (attributes are always private, and only in a few cases protected)
 - curly braces start at the end of a row
 - closing curly braces are to be written on a new row (after carriage return line feed)
 - avoid unnecessary empty rows
 - empty rows are to be used after package declaration, import statements, before classes, interfaces, and methods
 - Order of variables and methods within classes and interfaces:
 1. constants and variables (both called attributes)
 2. constructors
 3. getter and setter for attributes (only those which are necessary!)
 4. all other public methods
 5. further (protected and private) methods
- e) Each class and each interface is in a *separate* file.
(Exceptions are inline classes)
- f) At the top of each file you write: *authors, date, purpose of the class* and which *public-methods* exist for which purpose.

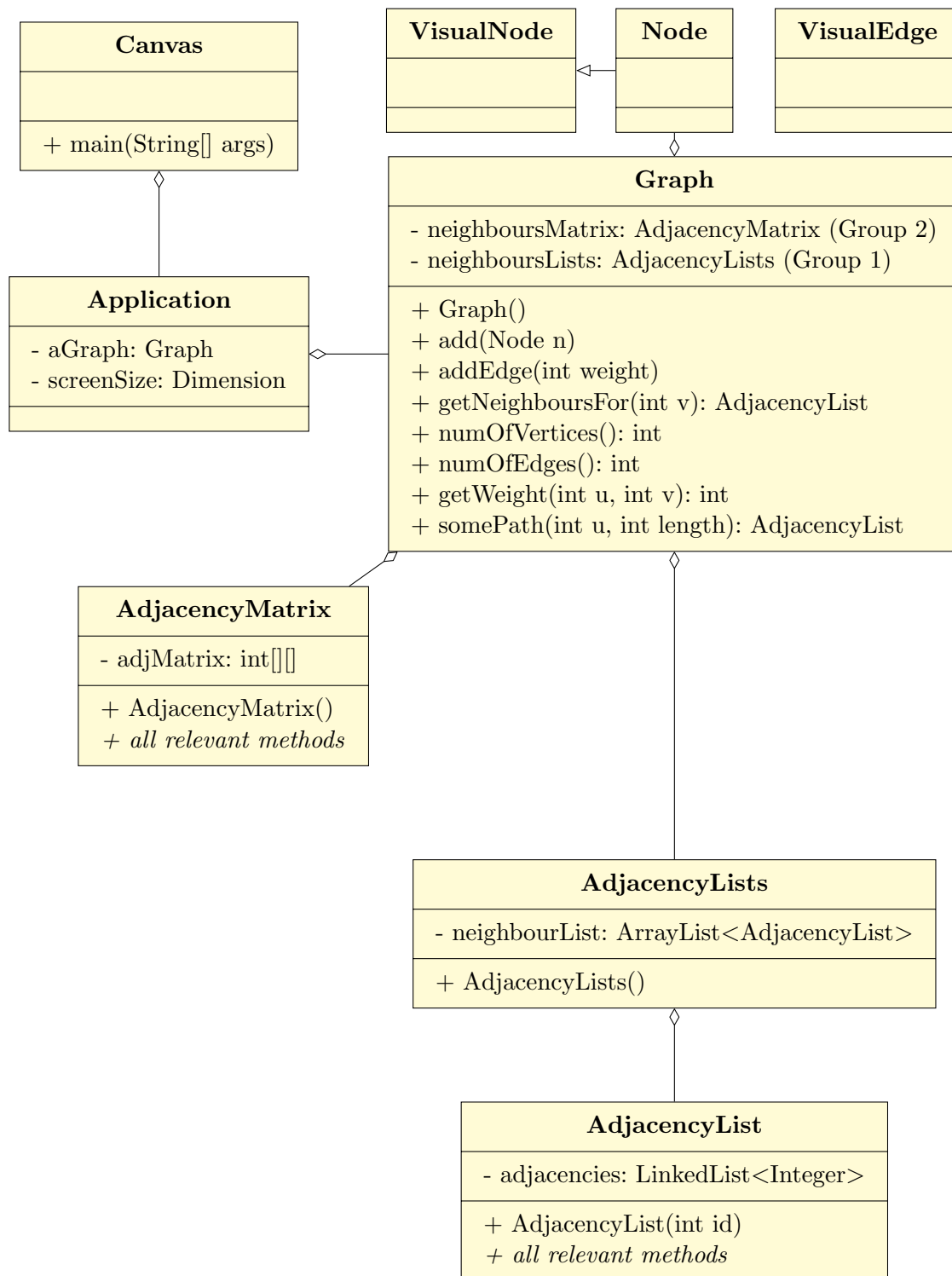


Abbildung 1: UML diagram for the classes to be implemented.